SOFTWARE REQUIREMENTS SPECIFICATION

for

ONLINE CREDIT-BASED E-LEARNING MANAGEMENT SYSTEM

Prepared by:

Student IDs: 23MCCE22 & 22MCCE21

Date: 3rd March 2026

SOFTWARE REQUIREMENTS SPECIFICATION(SRS)

1. INTRODUCTION

1.1 Purpose

This document provides a detailed description of the requirements for the **Online Credit Based E-Learning Management System (OCBELMS)**.

The purpose of this system is to digitally replicate the structured academic framework of an offline credit-based degree programme, including:

- Semester management
- Subject & credit allocation
- Student enrollment
- Continuous internal assessment
- Major examinations
- Grade calculation (SGPA/CGPA)
- Grade card generation

This SRS is intended for:

- Project developers
- Faculty evaluators
- Academic stakeholders
- System administrators

1.2 Scope

The system is designed for the **Department of Computer Science**, modeled for the University of Hyderabad. The system supports five user roles: **Admin, Course Coordinator, Academic Coordinator, Faculty, and Student**. It provides role-specific dashboards, enables semester-wise course management, credit allocation, online assessments, grade calculation (SGPA/CGPA), and grade card generation

The system will:

- Allow administrators to manage courses, coordinators, and user accounts.
- Enable course coordinators to define semesters, subjects, credits, and assign faculty.
- Support academic coordinators to register and enroll students.
- Allow faculty to upload learning materials, create assessments, enter marks, and generate grade cards.
- Allow students to access learning materials, attempt tests, and view their grades.

.

1.3 Definitions, Acronyms, Abbreviations:

Term / Acronym	Definition
SRS	Software Requirements Specification
OCBELMS	Online Credit-Based E-Learning Management System
Admin	System Administrator – manages top-level configuration
CC	Course Coordinator – manages academic structure and curriculum
AC	Academic Coordinator – manages student enrollment and records
SGPA	Semester Grade Point Average
CGPA	Cumulative Grade Point Average
UoH	University of Hyderabad
UC	Use Case
IEEE	Institute of Electrical and Electronics Engineers
HTML/CSS/JS	HyperText Markup Language / Cascading Style Sheets / JavaScript
MySQL	Open-source relational database management system
MVC	Model-View-Controller software architectural pattern

1.4 References

- **IEEE Std 830-1998** – IEEE Recommended Practice for Software Requirements Specifications..
- **Project Selection Document** – Online Credit-Based E-Learning Management System, Feb 2026.
- **Use Case Specifications Document** – OCBELMS, Feb 2026 (23MCCE22, 22MCCE21).
- University of Hyderabad Academic Regulations and Credit Framework.

1.5 Overall Description (Overview)

The system follows a **Three-Tier Architecture**:

1. Presentation Layer – Role-based dashboards
2. Application Layer – Business logic (credits, grading, enrollment)
3. Database Layer – MySQL/PostgreSQL

Technology Stack:

- Frontend: HTML, CSS, JavaScript, Bootstrap
- Backend: Java / PHP / Node.js (Express.js)

- Database: MySQL / PostgreSQL
- Tools: VS Code, GitHub, WAMP Server

The remaining sections of this SRS is organized as follows:

Section 2 – Overall Description: Describes the product perspective, functions, user characteristics, constraints, assumptions, and dependencies.

Section 3 – Specific Requirements: Contains detailed functional requirements through use case specifications, interface requirements, performance requirements, database requirements, design constraints, and software system attributes.

Section 4 – Supporting Information: Lists appendices and supplementary materials.

---

2. OVERALL DESCRIPTION

2.1 Product Perspective

The **OCBELMS** is a new, self-contained web-based platform. It does not replace any existing legacy system but is built from the ground up to digitally simulate the traditional offline academic system at the **University of Hyderabad.**

The system follows a three-tier architecture:

Presentation Layer: Role-based dashboards for Admin, Course Coordinator, Academic Coordinator, Faculty, and Student, built using HTML, CSS, JavaScript, and Bootstrap.

Application Layer: Server-side business logic handling credit calculations, grading rules, assessment management, and role-based access control, built using Java/PHP/Node.js with Express.js.

Database Layer: Persistent storage of all academic and user data using MySQL/PostgreSQL.

The system communicates via standard HTTP/HTTPS protocols. All user interactions happen through a web browser without any proprietary plugins.

2.2 Product Functions

Major functional modules:

User Role	Functionalities / Product Functions
Admin Module	• Create courses • Assign coordinators • Add users • Manage coordinators
Course Coordinator	• Define semesters • Add subjects • Assign credits • Assign faculty • Update curriculum
Academic Coordinator	• Register students • Enroll students • Assign student IDs • Manage student records

Faculty	• Upload lecture content • Create assignments/tests • Enter internal marks • Enter major marks • Generate grade card
Student	• View enrolled subjects • Access materials • Attempt tests • View marks • View grades

2.3 User Characteristics

Admin: Technically proficient staff responsible for system-level management. Expected to have familiarity with academic data management and system configuration.

Course Coordinator: Academic faculty or staff with strong knowledge of the university curriculum structure, subject-credit mappings, and faculty allocation.

Academic Coordinator: Administrative staff responsible for student enrollment and record-keeping. Requires basic computer literacy.

Faculty: Teaching staff with subject knowledge. Expected to have basic computer skills for uploading content, conducting assessments, and entering marks.

Student: Enrolled university students. Expected to have basic internet browsing skills and familiarity with online learning interfaces.

2.4 User Constraints

- All users must have a registered account with valid credentials before accessing the system.
- Role-based access control strictly limits each user's actions to their designated module.
- Students can only access subjects in which they are currently enrolled.
- Faculty can only manage subjects they are assigned to.
- Password policy must be enforced; accounts are locked after a maximum of 3 failed login attempts.
- Mark entry must be within predefined numeric ranges; violation triggers an error.
- Grade card generation is blocked until all marks (internal and major) are finalized.

2.5 . Assumptions and Dependencies

- Users have reliable internet access when interacting with the system.
- The hosting server (WAMP Server or equivalent) is operational and accessible.
- The MySQL/PostgreSQL database server is running and properly configured.
- The university's grading scheme (grade thresholds, SGPA/CGPA calculation formula) is fixed and provided before implementation.
- Course, semester, and subject structures are defined by the Course Coordinator before student enrollment begins.
- Faculty are registered in the system before the Course Coordinator attempts to assign them.
- Students possess valid documentation required for registration.

2.6. Apportioning of Requirements

The following requirements are identified as deferred to future versions of the system:

- Mobile application (Android/iOS) interface for students and faculty.
- Integration with a university-wide single sign-on (SSO) authentication service.
- Automated email/SMS notifications for marks publication and grade card generation.
- Advanced analytics and reporting dashboard for academic performance trends.
- Video conferencing or live class integration.
- Support for multiple departments or universities (multi-tenant architecture).

---

3. SPECIFIC REQUIREMENTS

3.1 Interface Requirements

3.1.1 External Interface

The system provides a web-based user interface accessible via standard browsers (Chrome, Firefox, Edge). All user interactions are performed through the browser. The external interface includes:

- Login page – accessible to all user types.
- Role-specific dashboards – displayed upon successful authentication.
- Forms for data entry (course creation, subject definition, mark entry, test submission).
- File upload interface for lecture content (Faculty module).
- Grade card display and print/download interface (Faculty and Student modules).

3.1.2 Hardware Interface

The system has no direct hardware interface requirements. It runs on standard web-capable devices (laptops, desktops, or tablets). Minimum requirements are:

- A device with an updated web browser.
- Internet connectivity (minimum 1 Mbps recommended for video content).
- Server hardware capable of running WAMP/LAMP stack with MySQL/PostgreSQL.

3.1.3 Software Interface

- Frontend: HTML5, CSS3, JavaScript, Bootstrap (optional).
- Backend: Java / PHP / Node.js with Express.js.
- Database: MySQL or PostgreSQL.
- Development Tools: VS Code, Git & GitHub, WAMP Server, MySQL Workbench.
- Operating System (Server): Windows (WAMP) or Linux (LAMP).

3.1.4 Communication Interface

- HTTP/HTTPS protocol for all client-server communication.
- Standard LAN or internet connectivity.
- Database communication via standard SQL over local connection or configurable DB host.

3.2 Functional Requirements:

3.2.1 USE CASE DIAGRAM /Information Flows-

This section presents the Use Case Model for the Online Credit-Based E-Learning Management System (OCBELMS). The use case model identifies all actors who interact with the system and all use cases (system behaviours) that satisfy the requirements of the project.

Actor	Description
Administrator	Responsible for top-level system management: creating courses, managing users, and assigning coordinators.
Course Coordinator	Defines academic structure: semesters, subjects, credits, and faculty assignment.
Academic Coordinator	Manages student lifecycle: registration, enrollment, ID assignment, and record management.
Faculty	Delivers teaching content, conducts assessments, enters marks, and generates grade cards.
Student	Consumes learning materials, attempts tests, and views academic performance records.

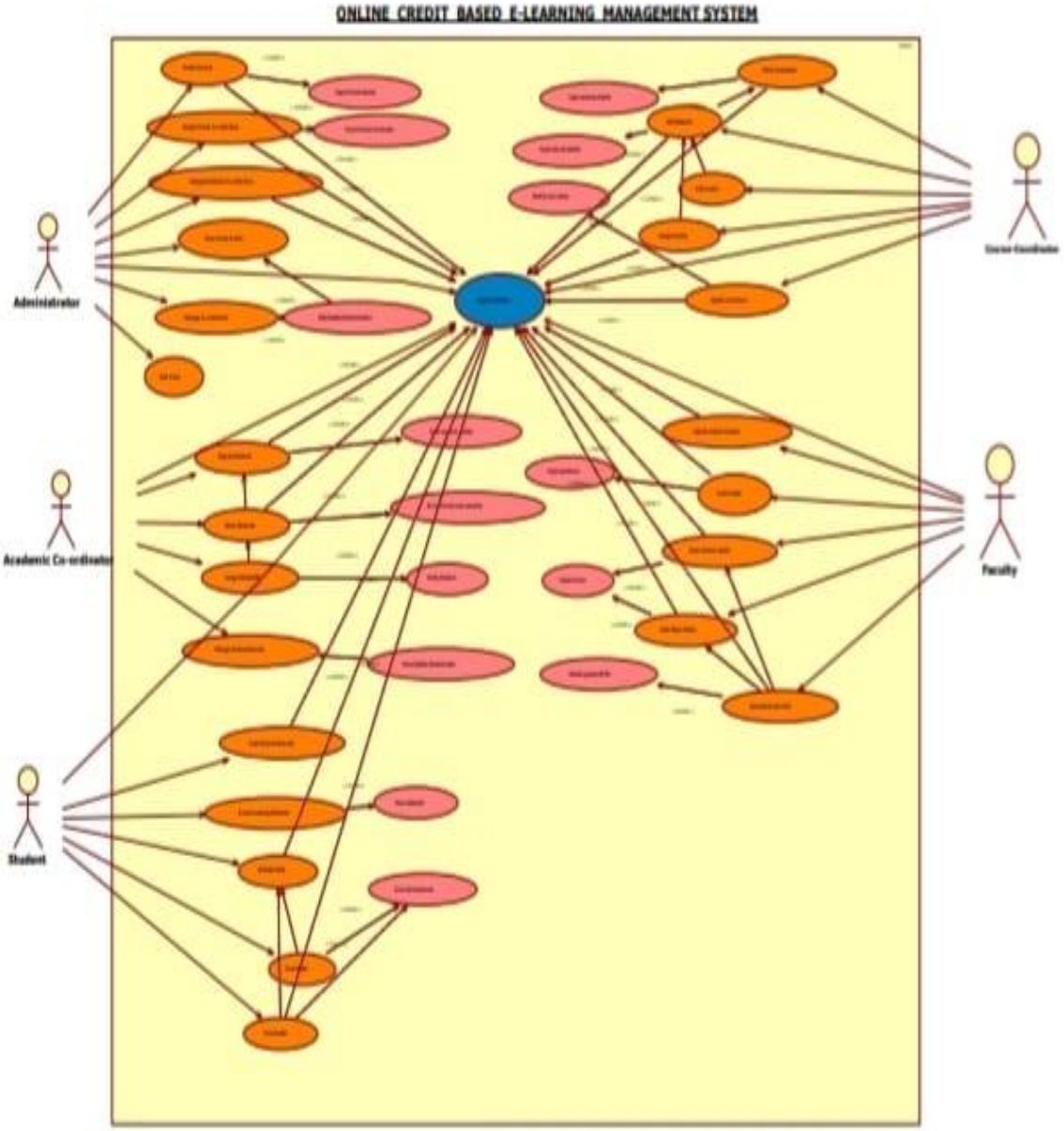
Use Case Relationships

- <<include>>: Mandatory sub-behaviour always executed as part of the base use case. For example, 'Create Courses' always includes 'Input Course Details'.
- <<extend>>: Optional behaviour executed under certain conditions. For example, 'Access Learning Materials' may extend to 'View and Download' when the student wishes to save materials offline.

Association: Each actor is connected to use cases they directly initiate.

Use Case Diagram

Figure 3.2.1 below presents the complete Use Case Diagram for the OCBELMS.



3.2.2 Use Case Specification/Process Descriptions:

Use Case – Login To System (Common to all actors)

Use Case ID	UC-001
Use Case Name	Login to System

Created By	Project Team
Date Created	25/02/2026
Last Updated By	Project Team
Date Last Updated	25/02/2026
Actors	Admin, Course Coordinator, Academic Coordinator, Faculty, Student
Description	Allows registered users to securely log into the Online Credit Based E-Learning Platform using valid credentials.
Preconditions	1. User must be registered in the system. 2. User must have valid login credentials (Username & Password).
Postconditions	1. User is successfully logged into the system. 2. Dashboard is displayed according to user role.
Normal Flow	1. User opens login page. 2. Enters username and password. 3. Clicks 'Login'. 4. System validates credentials. 5. System redirects user to respective dashboard.
Alternate Flow	1. User enters incorrect credentials. 2. System displays error message. 3. User re-enters credentials.
Exceptions	1. Account locked after multiple failed attempts. 2. Server unavailable.
Priority	High
Frequency of Use	Very Frequently
Business Rules	1. Password must follow security policy. 2. Maximum login attempts limited (e.g., 3 attempts).
Assumptions	1. User has internet access. 2. Authentication system is operational.

ACTOR – ADMIN

UC-01: Create Courses

Use Case ID	UC-01
Use Case Name	Create Courses

Created By	Project Team
Date Created	24-02-2026
Last Updated By	Project Team
Date Last Updated	24-02-2026
Actors	Administrator
Description	Admin creates a new course in the system.
Preconditions	Admin must be logged into the system.
Post Conditions	New course is successfully created and stored in database.
Normal Flow	1. Admin selects 'Create Course'. 2. System prompts for course details. 3. Admin enters details. 4. System saves course.
Alternate Flow	If details are incomplete, system shows error message.
Exceptions	Database failure or duplicate course ID.
Priority	High
Frequency of Use	Often
Business Rules	Course ID must be unique.
Assumptions	Admin has valid authorization.

UC-02: Assign Course Coordinators

Use Case ID	UC-02
Use Case Name	Assign Course Coordinator
Created By	Project Team
Date Created	24-02-2026
Last Updated By	Project Team
Date Last Updated	24-02-2026

Actors	Administrator
Description	Admin assigns a coordinator to a course.
Preconditions	Course must already exist.
Post Conditions	Coordinator is successfully assigned to course.
Normal Flow	1. Admin selects course. 2. System displays list of coordinators. 3. Admin selects coordinator. 4. System confirms assignment.
Alternate Flow	If coordinator already assigned, system shows message.
Exceptions	Invalid selection.
Priority	High
Frequency of Use	Often
Business Rules	One course must have at least one coordinator.
Assumptions	Coordinators are already registered.

UC-03: Assign Academic Coordinators

Use Case ID	UC-03
Use Case Name	Assign Academic Coordinator
Created By	Project Team
Date Created	24-02-2026
Last Updated By	Project Team
Date Last Updated	24-02-2026
Actors	Administrator
Description	Admin assigns academic coordinator to a course.
Preconditions	Course must exist.
Post Conditions	Academic coordinator linked to course.

Normal Flow	1. Admin selects course. 2. System displays academic coordinators. 3. Admin selects one. 4. System updates record.
Alternate Flow	If no coordinators available, show error.
Exceptions	System failure.
Priority	High
Frequency of Use	Sometimes
Business Rules	Only authorized users can be assigned.
Assumptions	Coordinators data is available.

UC-04: View Course Details

Use Case ID	UC-04
Use Case Name	View Course Details
Created By	Project Team
Date Created	24-02-2026
Last Updated By	Project Team
Date Last Updated	24-02-2026
Actors	Administrator
Description	Admin views details of a selected course.
Preconditions	Course must exist.
Post Conditions	Course details displayed.
Normal Flow	1. Admin selects course. 2. System displays course information.
Alternate Flow	If course not found, error shown.
Exceptions	System error.
Priority	Medium

Frequency of Use	Often
Business Rules	Only authorized users can view.
Assumptions	Course database is accessible.

UC-05: Manage Coordinators

Use Case ID	UC-05
Use Case Name	Manage Coordinators
Created By	Project Team
Date Created	24-02-2026
Last Updated By	Project Team
Date Last Updated	24-02-2026
Actors	Administrator
Description	Admin manages coordinator information.
Preconditions	Admin logged in.
Post Conditions	Coordinator information updated.
Normal Flow	1. Admin selects manage coordinators. 2. System displays list. 3. Admin updates details. 4. System saves changes.
Alternate Flow	Invalid data entered.
Exceptions	System failure.
Priority	High
Frequency of Use	Sometimes
Business Rules	Only valid data allowed.
Assumptions	Coordinator data exists.

UC-06: Add Users

Use Case ID	UC-06
Use Case Name	Add Users
Created By	Project Team
Date Created	24-02-2026
Last Updated By	Project Team
Last Updated Date	24-02-2026
Actors	Administrator
Description	Admin adds new users to the system.
Preconditions	Admin must be logged in.
Post Conditions	User successfully added.
Normal Flow	1. Admin selects Add User. 2. System prompts for user details. 3. Admin enters data. 4. System saves user.
Alternate Flow	If user ID already exists, show error.
Exceptions	Database failure.
Priority	High
Frequency of Use	Often
Business Rules	User ID must be unique.
Assumptions	Admin has permission.

ACTOR – COURSE COORDINATOR

UC-07: Define Semesters

Use Case ID	UC-07
Use Case Name	Define Semester

Created By	Project Team
Date Created	24-02-2026
Last Updated By	Project Team
Date Last Updated	24-02-2026
Actors	Course Coordinator
Description	Coordinator defines semester details in the system.
Preconditions	Coordinator must be logged in.
Post Conditions	Semester details stored successfully.
Normal Flow	1. Select Define Semester. 2. Enter semester details. 3. System saves data.
Alternate Flow	If fields are empty, error shown.
Exceptions	System/database failure.
Priority	High
Frequency of Use	Once per semester
Business Rules	Semester code must be unique.
Assumptions	Academic year already exists.

UC-08: Add Subjects

Use Case ID	UC-08
Use Case Name	Add Subjects
Created By	Project Team
Date Created	24-02-2026
Last Updated By	Project Team
Date Last Updated	24-02-2026

Actors	Course Coordinator
Description	Adds subjects under a semester.
Preconditions	Semester must be defined.
Post Conditions	Subjects added successfully.
Normal Flow	1. Select Add Subjects. 2. Enter subject details. 3. Save.
Alternate Flow	Duplicate subject code → error.
Exceptions	Invalid input.
Priority	High
Frequency of Use	Every semester
Business Rules	Subject code must be unique.
Assumptions	Semester exists.

UC-09: Add Credits

Use Case ID	UC-09
Use Case Name	Add Credits
Created By	Project Team
Date Created	24-02-2026
Last Updated By	Project Team
Date Last Updated	24-02-2026
Actors	Course Coordinator
Description	Assigns credit value to subjects.
Preconditions	Subject must exist.
Post Conditions	Credits updated successfully.

Normal Flow	1. Select subject. 2. Enter credit value. 3. Save.
Alternate Flow	Invalid credit value → error.
Exceptions	System failure.
Priority	High
Frequency of Use	Every semester
Business Rules	Credits must follow curriculum rules.
Assumptions	Subject already added.

UC-10: Assign Faculty

Use Case ID	UC-10
Use Case Name	Assign Faculty
Created By	Project Team
Date Created	24-02-2026
Last Updated By	Project Team
Date Last Updated	24-02-2026
Actors	Course Coordinator
Description	Assigns faculty to subjects.
Preconditions	Subject must exist.
Post Conditions	Faculty successfully assigned.
Normal Flow	1. Select subject. 2. Choose faculty. 3. Confirm assignment.
Alternate Flow	If faculty unavailable → show message.
Exceptions	Invalid selection.
Priority	High

Frequency of Use	Every semester
Business Rules	One subject must have assigned faculty.
Assumptions	Faculty list exists.

UC-11: Update Curriculum

Use Case ID	UC-11
Use Case Name	Update Curriculum
Created By	Project Team
Date Created	24-02-2026
Last Updated By	Project Team
Date Last Updated	24-02-2026
Actors	Course Coordinator
Description	Updates curriculum structure.
Preconditions	Curriculum must exist.
Post Conditions	Curriculum updated successfully.
Normal Flow	1. Select curriculum. 2. Modify details. 3. Save changes.
Alternate Flow	Invalid update → error shown.
Exceptions	System failure.
Priority	Medium
Frequency of Use	Occasionally
Business Rules	Must follow university regulations.
Assumptions	Coordinator has authority.

ACTOR – ACADEMIC COORDINATOR

UC-12: Register Student

Use Case ID	UC-12
Use Case Name	Register Student
Created By	Project Team
Date Created	24-02-2026
Actors	Academic Coordinator
Last Updated By	Project Team
Date Last Updated	24-02-2026
Description	Registers a new student in the system.
Preconditions	Coordinator must be logged in.
Post Conditions	Student basic record created.
Normal Flow	1. Select Register Student. 2. Enter student details. 3. Save information.
Alternate Flow	Missing details → error message.
Exceptions	Duplicate registration number.
Priority	High
Frequency of Use	Every academic year
Business Rules	Registration number must be unique.
Assumptions	Student documents are valid.

UC-13: Enroll Students

Use Case ID	UC-13
Use Case Name	Enroll Students
Created By	Project Team

Date Created	24-02-2026
Last Updated By	Project Team
Date Last Updated	24-02-2026
Actors	Academic Coordinator
Description	Enrolls registered student into course and semester.
Preconditions	Student must be registered.
Post Conditions	Student enrolled successfully.
Normal Flow	1. Select student. 2. Allocate course & semester. 3. Confirm enrollment.
Alternate Flow	Course full → error message.
Exceptions	System failure.
Priority	High
Frequency of Use	Every semester
Business Rules	Student must meet eligibility criteria.
Assumptions	Courses already defined.

UC-14: Assign Student ID

Use Case ID	UC-14
Use Case Name	Assign Student ID
Created By	Project Team
Date Created	24-02-2026
Last Updated By	Project Team
Date Last Modified	24-02-2026
Actors	Academic Coordinator

Description	Assigns unique ID to enrolled student.
Preconditions	Student must be enrolled.
Post Conditions	Unique Student ID generated.
Normal Flow	1. Select enrolled student. 2. Generate ID. 3. Save.
Alternate Flow	ID conflict → regenerate ID.
Exceptions	Database error.
Priority	High
Frequency of Use	Every enrollment
Business Rules	Student ID must be unique.
Assumptions	System auto-generates ID.

UC-15: Manage Student Records

Use Case ID	UC-15
Use Case Name	Manage Student Records
Created By	Project Team
Date Created	24-02-2026
Last Updated By	Project Team
Date Last Updated	24-02-2026
Actors	Academic Coordinator
Description	Manages and maintains student information.
Preconditions	Student record must exist.
Post Conditions	Updated record saved.
Normal Flow	1. Search student. 2. View details. 3. Update if required. 4. Save.

Alternate Flow	Invalid data entered → error.
Exceptions	Record not found.
Priority	Medium
Frequency of Use	Frequently
Business Rules	Only authorized updates allowed.
Assumptions	Coordinator has permission.

ACTOR – FACULTY

UC-16: Upload Lecture Content

Use Case ID	UC-16
Use Case Name	Upload Lecture Content
Created By	Project Team
Date Created	24/02/2026
Last Updated By	Project Team
Date Last Updated	24/02/2026
Actors	Faculty
Description	Faculty uploads lecture videos/notes for students.
Preconditions	Faculty must be logged in and course must be assigned.
Postconditions	Lecture content is stored and available to students.
Normal Flow	1. Select course. 2. Choose upload option. 3. Select file. 4. System validates file. 5. System uploads and confirms.
Alternate Flows	Invalid file format → system shows error.
Exceptions	Upload failure due to server/network issue.
Priority	High

Frequency of Use	Often
Business Rules	File size and format restrictions apply.
Assumptions	Faculty has valid course access.

UC-17: Create Tasks

Use Case ID	UC-17
Use Case Name	Create Tasks
Created By	Project Team
Date Created	24/02/2026
Last Updated By	Project Team
Date Last Updated	24/02/2026
Actors	Faculty
Description	Faculty creates assignments/tasks for students.
Preconditions	Faculty logged in and course selected.
Postconditions	Task is published to students.
Normal Flow	1. Select course. 2. Click create task. 3. Enter details. 4. Save task. 5. System confirms.
Alternate Flows	Missing required fields → prompt correction.
Exceptions	Database save failure.
Priority	High
Frequency of Use	Often
Business Rules	Deadline must be future date.
Assumptions	Course exists.

UC-18: Enter Internal Marks

Use Case ID	UC-18
Use Case Name	Enter Internal Marks
Created By	Project Team
Date Created	24/02/2026
Last Updated By	Project Team
Date Last Updated	24/02/2026
Actors	Faculty
Description	Faculty enters internal assessment marks.
Preconditions	Faculty logged in; students enrolled.
Postconditions	Internal marks saved.
Normal Flow	1. Select course. 2. Open internal marks. 3. Enter marks. 4. Save. 5. Confirmation displayed.
Alternate Flows	Marks out of range → error message.
Exceptions	Save failure/system crash.
Priority	High
Frequency of Use	Per assessment
Business Rules	Marks must be within allowed range.
Assumptions	Student list available.

UC-19: Enter Major Marks

Use Case ID	UC-19
Use Case Name	Enter Major Marks
Created By	Project Team

Date Created	24/02/2026
Last Updated By	Project Team
Date Last Updated	24/02/2026
Actors	Faculty
Description	Faculty enters major exam marks.
Preconditions	Faculty logged in; exam completed.
Postconditions	Major marks recorded.
Normal Flow	1. Select course. 2. Open major marks section. 3. Enter marks. 4. Save. 5. Confirmation displayed.
Alternate Flows	Invalid marks entered → prompt correction.
Exceptions	Database save error.
Priority	High
Frequency of Use	Once per semester
Business Rules	Marks cannot exceed maximum limit.
Assumptions	Students attempted exam.

UC-20: Generate Grade Card

Use Case ID	UC-20
Use Case Name	Generate Grade Card
Created By	Project Team
Date Created	24/02/2026
Last Updated By	Project Team
Date Last Updated	24/02/2026
Actors	Faculty

Description	Faculty generates final grade card.
Preconditions	Internal and major marks must be entered.
Postconditions	Grade card generated and stored.
Normal Flow	1. Select course. 2. Click generate grade card. 3. System calculates grades. 4. Grade card generated. 5. Confirmation shown.
Alternate Flows	Marks incomplete → generation blocked.
Exceptions	Calculation error.
Priority	High
Frequency of Use	Once per semester
Business Rules	Grade calculation follows university rules.
Assumptions	Marks are finalized and accurate.

ACTOR – STUDENT

UC-21: View Enrolled Subjects

Use Case ID	UC-21
Use Case Name	View Enrolled Subjects
Created By	Project Team
Date Created	24/02/2026
Last Updated By	Project Team
Date Last Updated	24/02/2026
Actors	Student
Description	Student views subjects allocated for the semester.
Preconditions	Student must be logged in and enrolled in a semester.
Postconditions	Enrolled subject list displayed.

Normal Flow	1. Student logs in. 2. Selects semester. 3. System displays enrolled subjects.
Alternate Flows	No subjects enrolled → system displays message.
Exceptions	Database retrieval failure.
Priority	High
Frequency of Use	Each semester
Business Rules	Student can view only allocated subjects.
Assumptions	Enrollment completed by Academic Coordinator.

UC-22: Access Learning Materials

Use Case ID	UC-22
Use Case Name	Access Learning Materials
Created By	Project Team
Date Created	24/02/2026
Last Updated By	Project Team
Date Last Updated	24/02/2026
Actors	Student
Description	Student accesses lecture materials uploaded by faculty.
Preconditions	Student logged in and enrolled in subject.
Postconditions	Materials displayed.
Normal Flow	1. Select subject. 2. Click access materials. 3. System retrieves content.
Alternate Flows	No materials uploaded.
Exceptions	File loading error.
Priority	High

Frequency of Use	Often
Business Rules	Access allowed only to enrolled subjects.
Assumptions	Faculty uploaded materials.

UC-23: Attempt Tests

Use Case ID	UC-23
Use Case Name	Attempt Tests
Created By	Project Team
Date Created	24/02/2026
Last Updated By	Project Team
Date Last Updated	24/02/2026
Actors	Student
Description	Student attempts online assessment.
Preconditions	Valid login and active test schedule.
Postconditions	Test responses submitted.
Normal Flow	1. Select test. 2. Answer questions. 3. Submit test.
Alternate Flows	Time expires → auto submit.
Exceptions	Network disconnection.
Priority	High
Frequency of Use	Per assessment
Business Rules	Single submission per test.
Assumptions	Test created by faculty.

UC-24: View Marks

Use Case ID	UC-24
Use Case Name	View Marks
Created By	Project Team
Date Created	24/02/2026
Last Updated By	Project Team
Date Last Updated	24/02/2026
Actors	Student
Description	Student views internal/exam marks.
Preconditions	Marks entered by faculty.
Postconditions	Marks displayed.
Normal Flow	1. Select subject. 2. Click view marks. 3. System displays marks.
Alternate Flows	Marks not yet published.
Exceptions	Data retrieval error.
Priority	High
Frequency of Use	Per assessment
Business Rules	Students cannot modify marks.
Assumptions	Faculty evaluated answers.

UC-25: View Grades

Use Case ID	UC-25
Use Case Name	View Grades
Created By	Project Team
Date Created	24/02/2026

Last Updated By	Project Team
Date Last Updated	24/02/2026
Actors	Student
Description	Student views subject-wise grades and SGPA/CGPA.
Preconditions	Grade card generated.
Postconditions	Grades displayed.
Normal Flow	1. Select semester. 2. Click view grades. 3. System displays grades.
Alternate Flows	Grades not yet published.
Exceptions	Calculation error.
Priority	High
Frequency of Use	Once per semester
Business Rules	Grade calculation follows predefined grading scheme.
Assumptions	Marks finalized by faculty.

3.2.3 Analysis Classes / Data Dictionary:

This section describes the analysis classes identified in the system using object-oriented analysis. Each analysis class is classified as one of the following types:

- Boundary Class: Represents the interface between actors and the system (screens, forms).
- Control Class: Contains the business logic and workflow coordination.

Entity Class: Represents persistent data or domain objects stored in the database.

A. Boundary Classes

LoginForm (Boundary)

Attributes	Operations / Methods	Associations
username : String **password : String** **errorMessage : String**	validateInput() submitLogin() displayError(msg) redirectToDashboard(role)	Interacts with: AuthController Used by: All Actors

AdminDashboard (Boundary)

Attributes	Operations / Methods	Associations
adminID : String menuOptions : List	displayCourses() displayUsers() navigateTo(module)	Interacts with: CourseController Interacts with: UserController

CoordinatorDashboard (Boundary)

Attributes	Operations / Methods	Associations
coordinatorID : String assignedCourses : List	displaySemesters() displaySubjects() navigateTo(module)	Interacts with: SemesterController Interacts with: SubjectController

FacultyDashboard (Boundary)

Attributes	Operations / Methods	**Associations**
facultyID : String assignedSubjects : List	displayContent() displayMarks() navigateTo(module)	Interacts with: ContentController Interacts with: MarksController

StudentDashboard (Boundary)

Attributes	Operations / Methods	Associations
studentID : String enrolledSubjects : List currentSemester : String	displaySubjects() displayGrades() navigateTo(module)	Interacts with: EnrollmentController Interacts with: GradeController

TestInterface (Boundary)

Attributes	Operations / Methods	Associations
testID : String questions : List timeRemaining : Integer answers : Map	loadQuestions() submitTest() autoSubmitOnTimeout() displayResult()	Interacts with: AssessmentController Used by: Student

B. Control Classes

AuthController (Control)

Attributes	Operations / Methods	Associations
maxAttempts : Integer = 3 sessionToken : String	authenticate(username, password) generateToken() invalidateSession() lockAccount(userID)	Uses: UserEntity Invokes: LoginForm

CourseController (Control)

Attributes	Operations / Methods	Associations
activeCourseID : String	createCourse(details) assignCoordinator(courseID, coordID) viewCourseDetails(courseID) listAllCourses()	Uses: CourseEntity Uses: UserEntity

SemesterController (Control)

Attributes	Operations / Methods	Associations
currentSemesterID : String	defineSemester(details) addSubject(semID, subjectDetails) addCredits(subjectID, credits) assignFaculty(subjectID, facultyID) updateCurriculum(semID, changes)	Uses: SemesterEntity Uses: SubjectEntity

EnrollmentController (Control)

Attributes	Operations / Methods	Associations
enrollmentStatus : String	registerStudent(details) enrollStudent(studentID, courseID, semID) assignStudentID(studentID) updateStudentRecord(studentID, data)	Uses: StudentEntity Uses: EnrollmentEntity

ContentController (Control)

Attributes	Operations / Methods	Associations
uploadStatus : String	uploadContent(subjectID, fileData) createTask(subjectID, taskDetails) validateFileFormat(file) listContent(subjectID)	Uses: LectureContentEntity Uses: AssessmentEntity

MarksController (Control)

Attributes	Operations / Methods	Associations
	enterInternalMarks(studentID, subjectID, marks) enterMajorMarks(studentID, subjectID, marks) validateMarksRange(marks, maxMarks) getMarks(studentID, subjectID)	Uses: MarksEntity Invokes: GradeController

GradeController (Control)

Attributes	Operations / Methods	Associations
gradingScheme : Map	calculateGrade(totalMarks, credits) computeSGPA(semesterMarks) computeCGPA(allSemesterMarks) generateGradeCard(studentID, semID)	Uses: MarksEntity Uses: GradeCardEntity Uses: SubjectEntity

AssessmentController (Control)

Attributes	Operations / Methods	Associations
activeTests : List	createTest(subjectID, questions) startTest(studentID, testID) submitTest(studentID, testID, answers) autoSubmit(testID) gradeTest(testID)	Uses: AssessmentEntity Uses: MarksEntity

C. Entity Classes

UserEntity (Entity)

Attributes	Operations / Methods	Associations
userID : String (PK) name : String email : String passwordHash : String role : Enum {Admin, CC, AC, Faculty, Student} status : Enum {Active, Locked}	getRole() isLocked() incrementFailedAttempts() resetAttempts()	Associated with: CourseEntity (Admin) Associated with: SubjectEntity (Faculty) Associated with: EnrollmentEntity (Student)

CourseEntity (Entity)

Attributes	Operations / Methods	Associations
courseID : String (PK) courseName : String departmentID : String courseCoordinatorID : String (FK) academicCoordinatorID : String (FK) status : Enum {Active, Inactive}	getCoordinator() getSemesters() isActive()	Has: SemesterEntity (1..*) Managed by: UserEntity (Admin)

SemesterEntity (Entity)

Attributes	Operations / Methods	Associations
semesterID : String (PK) courseID : String (FK) semesterNumber : Integer academicYear : String duration : String	getSubjects() getSemesterNumber()	Belongs to: CourseEntity Contains: SubjectEntity (1..*)

SubjectEntity (Entity)

Attributes	Operations / Methods	Associations
subjectID : String (PK) semesterID : String (FK) subjectName : String subjectCode : String credits : Integer type : Enum {Core, Elective, Lab} assignedFacultyID : String (FK)	getCredits() getFaculty() getEnrolledStudents()	Belongs to: SemesterEntity Taught by: UserEntity (Faculty)

StudentEntity (Entity)

Attributes	Operations / Methods	Associations
studentID : String (PK) registrationNo : String userID : String (FK) name : String dateOfBirth : Date contactInfo : String courseID : String (FK) currentSemesterID : String (FK)	getEnrolledSubjects() getMarks(subjectID) getGradeCard(semID)	Associated with: EnrollmentEntity Associated with: MarksEntity

EnrollmentEntity (Entity)

Attributes	Operations / Methods	Associations
enrollmentID : String (PK) studentID : String (FK) subjectID : String (FK) semesterID : String (FK)	isActive() getSubject()	Links: StudentEntity with SubjectEntity

Attributes	Operations / Methods	Associations
enrollmentDate : Date status : Enum {Active, Completed, Dropped}		

LectureContentEntity (Entity)

Attributes	Operations / Methods	Associations
contentID : String (PK) subjectID : String (FK) facultyID : String (FK) title : String fileURL : String fileType : String uploadDate : Date	getDownloadURL() isAccessibleTo(studentID)	Uploaded by: UserEntity (Faculty) Accessed by: StudentEntity

AssessmentEntity (Entity)

Attributes	Operations / Methods	Associations
assessmentID : String (PK) subjectID : String (FK) facultyID : String (FK) type : Enum {Internal, Major, Task} title : String maxMarks : Integer deadline : DateTime questions : List	isActive() getQuestions() isSubmitted(studentID)	Created by: UserEntity (Faculty) Attempted by: StudentEntity

MarksEntity (Entity)

Attributes	Operations / Methods	Associations
markID : String (PK) studentID : String (FK) subjectID : String (FK) semesterID : String (FK) internalMarks : Float majorMarks : Float totalMarks : Float isPublished : Boolean	getTotalMarks() isComplete() publish()	Associated with: StudentEntity Used by: GradeController

GradeCardEntity (Entity)

Attributes	Operations / Methods	Associations
gradeCardID : String (PK) studentID : String (FK) semesterID : String (FK) subjectGrades : Map SGPA : Float CGPA : Float generatedDate : Date isPublished : Boolean	getSGPA() getCGPA() getGradeForSubject(subjectID) publish()	Generated by: GradeController Viewed by: StudentEntity

.

Data Dictionary

The data dictionary defines all data elements used across the system, including their types, sizes, constraints, and descriptions.

Table / Entity: users

Attribute Name	Data Type	Size	Constraints	Description
user_id	VARCHAR	20	PK, NOT NULL	Unique identifier for each user
name	VARCHAR	100	NOT NULL	Full name of the user
email	VARCHAR	150	UNIQUE, NOT NULL	Email address used for login
password_hash	VARCHAR	255	NOT NULL	Bcrypt-hashed password
role	ENUM	-	NOT NULL	Admin | CourseCoord | AcademicCoord | Faculty | Student
status	ENUM	-	DEFAULT Active	Active | Locked
created_at	DATETIME	-	NOT NULL	Timestamp of account creation
failed_attempts	INT	1	DEFAULT 0	Consecutive failed login attempts

Table / Entity: courses

Attribute Name	Data Type	Size	Constraints	Description
course_id	VARCHAR	20	PK, NOT NULL	Unique course identifier
course_name	VARCHAR	150	NOT NULL	Full name of the course
department_id	VARCHAR	20	NOT NULL	Department to which course belongs
course_coordinator_id	VARCHAR	20	FK -> users.user_id	Assigned Course Coordinator

Attribute Name	Data Type	Size	Constraints	Description
academic_coordinator_id	VARCHAR	20	FK -> users.user_id	Assigned Academic Coordinator
status	ENUM	-	DEFAULT Active	Active | Inactive

Table / Entity: semesters

Attribute Name	Data Type	Size	Constraints	Description
semester_id	VARCHAR	20	PK, NOT NULL	Unique semester identifier
course_id	VARCHAR	20	FK -> courses.course_id	Parent course
semester_number	INT	2	NOT NULL, > 0	Semester number (1 to 8)
academic_year	VARCHAR	10	NOT NULL	e.g., 2025-26
duration	VARCHAR	50	NOT NULL	e.g., July 2025 to November 2025

Table / Entity: subjects

Attribute Name	Data Type	Size	Constraints	Description
subject_id	VARCHAR	20	PK, NOT NULL	Unique subject identifier
semester_id	VARCHAR	20	FK -> semesters.semester_id	Parent semester
subject_name	VARCHAR	150	NOT NULL	Full subject name
subject_code	VARCHAR	20	UNIQUE, NOT NULL	Unique subject code (e.g., CS501)
credits	INT	2	NOT NULL, > 0	Credit value of the subject
type	ENUM	-	NOT NULL	Core | Elective | Lab
faculty_id	VARCHAR	20	FK -> users.user_id	Assigned faculty member

Table / Entity: students

Attribute Name	Data Type	Size	Constraints	Description
student_id	VARCHAR	20	PK, NOT NULL	System-generated unique student ID
registration_no	VARCHAR	30	UNIQUE, NOT NULL	University registration number
user_id	VARCHAR	20	FK -> users.user_id	Linked user account
name	VARCHAR	100	NOT NULL	Full name of the student
date_of_birth	DATE	-	NOT NULL	Student date of birth

Attribute Name	Data Type	Size	Constraints	Description
contact_info	VARCHAR	255	-	Phone and address details
course_id	VARCHAR	20	FK -> courses.course_id	Enrolled course
current_semester_id	VARCHAR	20	FK -> semesters.semester_id	Current active semester

Table / Entity: enrollments

Attribute Name	Data Type	Size	Constraints	Description
enrollment_id	VARCHAR	20	PK, NOT NULL	Unique enrollment record ID
student_id	VARCHAR	20	FK -> students.student_id	Student being enrolled
subject_id	VARCHAR	20	FK -> subjects.subject_id	Subject enrolled in
semester_id	VARCHAR	20	FK -> semesters.semester_id	Semester of enrollment
enrollment_date	DATE	-	NOT NULL	Date of enrollment
status	ENUM	-	DEFAULT Active	Active | Completed | Dropped

Table / Entity: lecture_content:

Attribute Name	Data Type	Size	Constraints	Description
content_id	VARCHAR	20	PK, NOT NULL	Unique content identifier
subject_id	VARCHAR	20	FK -> subjects.subject_id	Subject to which content belongs
faculty_id	VARCHAR	20	FK -> users.user_id	Faculty who uploaded the content
title	VARCHAR	200	NOT NULL	Title of the content
file_url	VARCHAR	500	NOT NULL	Path or URL of uploaded file
file_type	VARCHAR	20	NOT NULL	PDF | MP4 | DOCX | URL
upload_date	DATETIME	-	NOT NULL	Timestamp of upload

Table / Entity: assessments

Attribute Name	Data Type	Size	Constraints	Description
assessment_id	VARCHAR	20	PK, NOT NULL	Unique assessment identifier

Attribute Name	Data Type	Size	Constraints	Description
subject_id	VARCHAR	20	FK -> subjects.subject_id	Subject for this assessment
faculty_id	VARCHAR	20	FK -> users.user_id	Faculty who created the assessment
type	ENUM	-	NOT NULL	Internal | Major | Task
title	VARCHAR	200	NOT NULL	Title of assessment
max_marks	INT	3	NOT NULL, > 0	Maximum marks
deadline	DATETIME	-	NOT NULL	Submission deadline (must be future)

Table / Entity: marks

Attribute Name	Data Type	Size	Constraints	Description
mark_id	VARCHAR	20	PK, NOT NULL	Unique marks record ID
student_id	VARCHAR	20	FK -> students.student_id	Student for whom marks are recorded
subject_id	VARCHAR	20	FK -> subjects.subject_id	Subject for which marks are entered
semester_id	VARCHAR	20	FK -> semesters.semester_id	Semester context
internal_marks	DECIMAL	5,2	0 <= value <= max	Internal/continuous assessment marks
major_marks	DECIMAL	5,2	0 <= value <= max	End-semester examination marks
total_marks	DECIMAL	5,2	Computed field	Sum of internal and major marks
is_published	BOOLEAN	-	DEFAULT FALSE	Whether marks are visible to student

Table / Entity: grade_cards

Attribute Name	Data Type	Size	Constraints	Description
grade_card_id	VARCHAR	20	PK, NOT NULL	Unique grade card identifier
student_id	VARCHAR	20	FK -> students.student_id	Student for whom card is generated
semester_id	VARCHAR	20	FK -> semesters.semester_id	Semester of the grade card
sgpa	DECIMAL	4,2	0.00 to 10.00	Semester Grade Point Average

Attribute Name	Data Type	Size	Constraints	Description
cgpa	DECIMAL	4,2	0.00 to 10.00	Cumulative Grade Point Average
generated_date	DATETIME	-	NOT NULL	Timestamp when card was generated
is_published	BOOLEAN	-	DEFAULT FALSE	Whether card is visible to student

Table / Entity: subject_grades

Attribute Name	Data Type	Size	Constraints	Description
grade_entry_id	VARCHAR	20	PK, NOT NULL	Unique entry per subject grade
grade_card_id	VARCHAR	20	FK -> grade_cards.grade_card_id	Parent grade card
subject_id	VARCHAR	20	FK -> subjects.subject_id	Subject for this grade entry
grade	ENUM	-	NOT NULL	O | A+ | A | B+ | B | C | D | F
grade_points	DECIMAL	4,2	0.00 to 10.00	Grade points for the subject
credits_earned	INT	2	>= 0	Credits earned (0 if grade is F)

Grading Scheme Reference

Grade	**Marks Range (%)**	**Grade Points**	**Performance**
O (Outstanding)	90 and above	10.0	Outstanding
A+ (Excellent)	80 – 89	9.0	Excellent
A (Very Good)	70 – 79	8.0	Very Good
B+ (Good)	60 – 69	7.0	Good
B (Above Average)	55 – 59	6.0	Above Average
C (Average)	50 – 54	5.0	Average
D (Pass)	40 – 49	4.0	Pass
F (Fail)	Below 40	0.0	Fail – Repeat Required

SGPA Formula: SGPA = Sum (Grade Points x Credits) / Sum (Credits) for all subjects in the semester.

CGPA Formula: CGPA = Sum (Grade Points x Credits) / Sum (Credits) across all completed semesters.

3.3 Performance Requirements

1. The system shall support at least **500 concurrent users** without degradation of performance.
2. The system shall complete **user login validation within 3 seconds** under normal network conditions.
3. The system shall generate and display the **grade card within 5 seconds** after the request is initiated.
4. The system shall allow file uploads with a **configurable maximum file size (e.g., 50MB)** for lecture materials.
5. The system shall load dashboard pages within **3–5 seconds** for all user roles under standard server load.
6. The system shall process and store assessment marks for an entire class (minimum 60 students) within **5 seconds**.
7. The system shall support simultaneous test submissions from at least **200 students** without data loss.
8. The system shall ensure database query response time does not exceed **2 seconds** for common operations such as viewing subjects or marks.
9. The system shall maintain **99% system availability** during academic working hours.
10. The system shall perform automated grade calculations for an entire semester within **10 seconds**.

3.4 Logical Database Requirements

The system shall use a relational database (MySQL/PostgreSQL) to store academic, user, and assessment data. The database shall maintain data integrity using primary keys, foreign keys, and constraints.

The database must maintain the following primary entities and their relationships:

3.4.1 Table Relationship Description:

Table / Entity	Key Attributes
users	user_id (PK), name, email, password_hash, role, status
courses	course_id (PK), course_name, department_id, course_coordinator_id (FK), academic_coordinator_id (FK)

semesters	semester_id (PK), course_id (FK), semester_number, academic_year, duration
subjects	subject_id (PK), semester_id (FK), subject_name, subject_code, credits, type, faculty_id (FK)
students	student_id (PK), registration_no, user_id (FK), name, course_id (FK), current_semester_id (FK)
enrollments	enrollment_id (PK), student_id (FK), subject_id (FK), semester_id (FK), status
lecture_content	content_id (PK), subject_id (FK), faculty_id (FK), title, file_url, upload_date
assessments	assessment_id (PK), subject_id (FK), faculty_id (FK), type, max_marks, deadline
marks	mark_id (PK), student_id (FK), subject_id (FK), internal_marks, major_marks, total_marks
grade_cards	grade_card_id (PK), student_id (FK), semester_id (FK), sgpa, cgpa, generated_date
subject_grades	grade_entry_id (PK), grade_card_id (FK), subject_id (FK), grade, grade_points, credits_earned

3.4.3 Data Integrity Constraints

- Course ID, Semester ID, Subject ID, Student ID must be **unique**.
- Credit values must be **positive numeric values**.
- Marks must be within the **defined maximum limit**.
- Foreign key constraints shall maintain referential integrity.
- Deletion of a course shall not occur if dependent semester records exist.

3.4.4 Database Design Approach

The database design follows:

- Relational Model
- Normalization up to Third Normal Form (3NF)
- Primary & Foreign Key constraints
- Role-Based Access Control enforcement

3.5 Design Constraints

The system design shall adhere to the following constraints based on the project requirements:

1. The system must strictly follow the **University-approved grading scheme** for calculating subject grades, SGPA, and CGPA.
2. The system must implement a **credit-based academic structure**, where grade calculation depends on subject credits.
3. The system shall enforce **role-based access control (RBAC)** to restrict functionalities based on user roles (Admin, Course Coordinator, Academic Coordinator, Faculty, Student).
4. All entities such as Course ID, Semester ID, Subject Code, Faculty ID, and Student ID must be **unique** within the system.

5. The system must follow a **three-tier architecture** (Presentation Layer, Application Layer, Database Layer) as defined in the project design.
6. The database must maintain **referential integrity** using primary and foreign key constraints.
7. The system shall be designed as a **web-based application**, compatible with modern browsers (Chrome, Firefox, Edge).
8. The system shall use **relational database technology (MySQL/PostgreSQL)** for structured academic data management.
9. The system must restrict students to access **only enrolled subjects and related materials**.
10. The system must limit test attempts to **single submission per scheduled assessment**.
11. The grade card generation logic must not allow modification once finalized, except by authorized personnel.
12. The system must comply with basic **data security practices**, including password encryption and secure authentication.

3.6 Software System Attributes

3.6.1 Reliability

- **Data consistency ensured through DB constraints**
  The system shall use primary keys, foreign keys, and validation rules to prevent inconsistent or invalid data entries.
- **Auto-save mechanisms for marks entry**
  The system shall automatically save entered marks periodically to prevent data loss due to unexpected system failure.

3.6.2 Availability

- **System available 24/7**
  The system shall be accessible to users at all times except during scheduled maintenance periods.
- **Downtime only during maintenance**
  The system shall minimize downtime and notify users in advance of any planned maintenance activities.

3.6.3 Security

- **Password encryption**
  The system shall store user passwords in encrypted or hashed format to prevent unauthorized access.
- **Role-based access control**
  The system shall restrict access to features and data based on the assigned user role.
- **Max login attempts limit**
  The system shall lock user accounts temporarily after a predefined number of failed login attempts.
- **Secure session handling**
  The system shall maintain secure user sessions using session tokens and automatic logout after inactivity.

3.6.4 Maintainability

- **Modular architecture**
  The system shall be divided into independent modules to simplify debugging and future enhancements.
- **Separated business logic**
  The system shall maintain separation between presentation, application logic, and database layers for easier updates.
- **Scalable database design**
  The database shall support future expansion such as additional courses, departments, or user growth.

3.6.5 Portability

- **Web-based; works on any browser**
  The system shall operate on standard web browsers without requiring additional software installation.
- **Deployable on Windows/Linux servers**
  The system shall be compatible with both Windows and Linux server environments for flexible deployment.

4.SUPPORTING INFORMATION –

4.1 Appendix A - System Workflow Summary

Summarised roles and responsibilities of each actor:

Actor	Responsibility
Admin	Creates courses; assigns Course Coordinator and Academic Coordinator; manages users.
Course Coordinator	Defines semesters; adds subjects and credits; assigns faculty; updates curriculum.
Academic Coordinator	Registers and enrolls students; assigns Student IDs; manages student records.
Faculty	Uploads lecture content; creates tasks; enters internal and major marks; generates grade cards.
Student	Views enrolled subjects; accesses learning materials; attempts tests; views marks, grades, SGPA/CGPA.

4.2 Appendix B - Technology Stack

Core technologies used to build and run the platform:

Layer	Technology
Frontend	HTML5, CSS3, JavaScript, Bootstrap

Backend	Java / PHP / Node.js with Express.js
Database	MySQL / PostgreSQL
Architecture	Three-tier: Presentation / Application / Database
Dev Tools	VS Code, WAMP Server, MySQL Workbench, Git & GitHub

4.4 Document Prepared By:

Name / ID	Role	Date
23MCCE22	Project Team	25-02-2026
22MCCE21	Project Team	25-02-2026

— End of Software Requirements Specification —